Report v27

AR-Enhanced Maintenance Support System

A TRL-3 Prototype for Bournemouth Central Bus Depot

COMP5067 Technological Innovations in Computing Bournemouth University, Level 5

Group members: Jude, Abishek, Shiar, Exauce, Mikku **Submission date:** May 2026 **Version:** 27

Abstract

This project presents a Technology Readiness Level 3 prototype of an Augmented Reality (AR) maintenance support system aimed at Bournemouth Central Bus Depot. The system combines marker-based AR inspection with a machine learning failure risk model, role-based access control with bcrypt password hashing and signed JSON Web Tokens, an offline-capable progressive web app shell, a hash-chain audit log for tool movement, and a supervisor analytics dashboard. A logistic regression model trained on 5,000 synthetic records achieved an F1 score of 0.850 and an AUC-ROC of 0.926. The prototype is deployed on Vercel and verified by 26 automated tests covering both ML inference and the REST API. The work demonstrates how low-cost AR and explainable ML can support fleet maintenance while remaining within the resource limits of a small depot.

Keywords: augmented reality, predictive maintenance, logistic regression, SHAP, JWT, bcrypt, accessibility, TRL-3.

1. Introduction

Public transport depots in the UK face rising maintenance demand on ageing fleets while skilled labour shortages limit inspection throughput. Bournemouth Central Bus Depot reported in 2024 that nearly 40 per cent of brake-related faults were detected only after a driver complaint rather than during scheduled inspection. The cost of unplanned downtime is significant for both the operator and the passenger.

This project addresses that gap with a prototype maintenance support system that lets mechanics scan an AR marker beside a component to see overlaid fault data and a predicted failure probability for the next 30 days. Supervisors view aggregated risk across the fleet from a separate analytics dashboard.

The core research question is whether a low-cost browser-based prototype can demonstrate the user-facing flows of an AR plus ML maintenance system at TRL-3 without requiring custom AR hardware or a deployed cloud database. The contribution of this work is a fully working frontend, a real Node.js backend with bcrypt password hashing and JWT authentication, a real trained logistic regression model with SHAP explainability, and a published deployment artefact.

1.1 Aims and objectives

The aims of the prototype are to:

1. Reduce the time taken to retrieve maintenance history for a specific vehicle component during inspection.
2. Surface the highest risk vehicles to supervisors before failures occur in service.
3. Provide an auditable record of tool movement to address the report by the depot manager that hand tools were going missing.
4. Meet WCAG 2.1 AA accessibility standards so the interface is usable by mechanics with low-vision needs.

1.2 Stakeholders

The primary stakeholders are mechanics, who use the AR view daily, and supervisors, who use the dashboard. Secondary stakeholders include the depot manager, who is responsible for compliance and audit, and senior management at the operator who fund the system. The brief positioned all five group members as independent contributors with named ownership of subsystems, described in section 8.

2. Background and Related Work

2.1 Maintenance in transport

Predictive maintenance has been studied in aviation and rail for over two decades. The dominant approach pairs sensor telemetry with statistical or machine learning models. For a bus depot the constraint is that retrofitting telemetry to older vehicles is expensive, so this prototype uses historical fault records as the primary input rather than live sensor streams. This choice trades off predictive ceiling for deployment cost, which is appropriate at TRL-3.

2.2 AR for industrial inspection

Marker-based AR is mature enough for industrial use. Boeing, Lockheed Martin and Volkswagen have all reported reductions in error rates of 20 to 40 per cent when overlaying assembly or inspection instructions through head-worn displays. The cost of head-worn hardware remains a barrier for small depots, so this prototype uses a tablet-style camera view with the AR.js library. This pushes AR onto commodity Android tablets that the depot already owns.

2.3 Explainable ML

Logistic regression remains the workhorse of operational ML because its coefficients are directly interpretable. The SHAP library extends interpretability to per-prediction attributions and was used here for the feature importance figure. Random forest and gradient boosting models were considered but rejected for this prototype because their additional accuracy did not outweigh the loss of interpretability at TRL-3.

2.4 Accessibility and inclusive design

The depot employs at least one technician with reduced colour vision. WCAG 2.1 AA was chosen as the accessibility target because it is the public-sector default in the UK under the Public Sector Bodies Accessibility Regulations 2018. Two switchable themes were implemented to meet this target.

3. Requirements

3.1 Functional requirements

The prototype must:

- Allow a mechanic to log in with a username and password, with passwords stored as bcrypt hashes rather than plaintext.
- Allow a mechanic to scan a printed AR marker and see fault data overlaid on the camera view.
- Display a 30-day predicted failure probability for the selected record.
- Allow tool checkout and return with an immutable audit trail.
- Allow a supervisor to view fleet-wide analytics including SHAP explanations and a confusion matrix for model performance.
- Continue to function offline after the first visit.

3.2 Non-functional requirements

The prototype must:

- Respond to a prediction request in under 100 ms at the 95th percentile.
- Meet WCAG 2.1 AA across all pages.
- Pass an automated test suite of at least 20 tests on each commit.

- Be deployable to a free hosting tier so the examiner can view the live system from a browser.

3.3 Out of scope

The TRL-3 build does not include real telemetry ingestion, OAuth federation with depot identity providers, MongoDB persistence, custom AR marker training, or head-worn display support. These are mapped to TRL-6 in section 7.

4. System Design

4.1 Architecture overview

The system is split into three runtime tiers. The frontend is a pure ES module single-page application served as static files. The backend is an Express application that exposes a REST API and is deployed as a serverless function. The ML logic is executed inside the backend with coefficients trained offline in Python. There is no separate database tier at TRL-3. State persists in an in-memory store on the backend and falls back to the browser localStorage when the backend is unreachable. This dual-store design lets the app remain interactive during a depot Wi-Fi outage.

```
[Mechanic Tablet]  ----HTTPS---->  [Vercel Edge]
       |                                |
   AR.js + A-Frame                  Express API
   Service Worker                   bcrypt + JWT
   Chart.js dashboard               LR /predict
       |                                |
   localStorage fallback           In-memory store
```

4.2 Module breakdown

The frontend code is broken into seven ES modules. Each module has a single responsibility and exports a small surface so unit testing is straightforward. The modules are api.js for HTTP plus fallback, mlService.js for local inference, dashboard.js for charts, ar.js for marker control, toolcheck.js for the audit log, anomaly.js for the rule-based monitoring panel, and accessibility.js for the contrast and text-size toggles.

4.3 Data model

A maintenance record has the fields id, type, title, location, severity, status, notes, inspectionNotes, createdBy, and createdAt. Severity is an enum of low, medium, high, critical. Status is open, inspected, missing, or returned. The data model was kept deliberately narrow because adding fields tends to push the form into being slower to complete on a tablet keyboard, which directly conflicts with objective 1 in section 1.

5. Implementation

5.1 Frontend

The frontend uses no build step. ES modules are loaded directly by the browser via `<script type="module">`, which means the project can run on any static host without a bundler. Chart.js 4 is loaded from a CDN for the analytics dashboard. A-Frame 1.4 plus AR.js 3.4 power the AR view. Two markers are configured. The Hiro marker triggers a blue cube overlay used for fault records. The Kanji marker triggers a purple cylinder used for tool records. Splitting the markers by record type was a deliberate design decision to reduce mechanic cognitive load when both record types are visible in the same bay.

5.2 Backend

The backend is an Express 4 application written in Node.js 22. It exposes the following routes:

- `POST /auth/login` issues a signed HS256 JWT after verifying the supplied password against the bcrypt hash for that user.
- `GET /v1/items` returns all maintenance records, protected by JWT.
- `POST /v1/items` creates a new record.
- `POST /v1/items/:id/inspect` appends an inspection note and updates the status.

- `POST /v1/items/:id/move` records a tool checkout or return.
- `POST /v1/reset` resets the store. Admin role only.
- `POST /predict` returns a 30-day failure probability and risk band.
- `GET /health` reports model metadata and uptime.

Routes use the `/v1/` prefix rather than `/api/` to avoid colliding with the Vercel platform convention of using the `/api/` folder for serverless functions.

5.3 Authentication and authorisation

Login requires a username and password. Passwords are stored as bcrypt hashes with a cost factor of 10. Three seed accounts are provided. On successful match the backend returns a JWT signed with HS256 and an eight-hour expiry. The token payload carries name and role claims. Every protected route validates the token with `jwt.verify` and reads the role claim. Admin-only routes return 403 if the role is not admin. This is a simple but functional implementation of role-based access control. A TRL-6 upgrade would replace bcrypt with argon2id, add a refresh token, and store users in MongoDB.

5.4 Machine learning pipeline

The ML model is a scikit-learn logistic regression trained on 5,000 synthetic records generated by `train_model.py`. The feature vector for each record is severity, component type, bus age, mileage, days since last service, and open status. Class weights were set to balanced to compensate for the natural imbalance between healthy and faulty records. The trained intercept and six coefficients are embedded as constants in both `backend/app.js` and `js/mlService.js`. This duplication means the frontend can still produce a prediction when the backend is unreachable, using the same maths.

The trained model achieved the following test metrics on a 1,250 record holdout:

Metric	Value
F1 score	0.850
AUC-ROC	0.926
Recall	0.835
Precision	0.865

The choice of logistic regression rather than a tree ensemble is justified in section 2.3. Per-prediction explanations are produced by SHAP. The SHAP linear explainer was applied to the test set and the mean absolute SHAP values per feature are visualised on the dashboard. Severity is the dominant feature with status as a strong secondary signal.

Figure 1. Confusion matrix for the trained model on 1,250 test samples.

	Predicted Failure	Predicted Healthy
Actual Failure	563 (TP)	111 (FN)
Actual Healthy	88 (FP)	488 (TN)

The false negative rate is 16.5 per cent. This is the most operationally costly cell because a missed failure can lead to a vehicle being put into service while unsafe. The threshold could be lowered to reduce FN at the cost of FP, but this is left for a future stakeholder review with the depot.

5.5 Drift monitoring

A Population Stability Index is computed at each ping. PSI compares the expected feature distribution at training time against the observed feature distribution at inference time. A PSI under 0.10 is treated as stable, 0.10 to 0.20 as warning, and over 0.20 as a retrain trigger. The dashboard displays the live PSI with a colour-coded badge. At TRL-3 the PSI value is simulated. At TRL-6 it would be computed from a rolling window of real predictions.

5.6 Tool checkout with hash-chain audit log

Tool movements are recorded as a chained log. Each entry hashes the previous entry plus the new entry fields. Editing any entry breaks the chain hash for every later entry, which makes the log tamper-evident. The hash function at TRL-3 is a simple non-cryptographic hash for demo speed. At TRL-6 it would be SHA-256 via the Web Crypto API. The dashboard shows the live chain hash prefix and a status badge.

5.7 Anomaly detection

Three rule-based detectors run on every page load. D1 flags sessions where the action count exceeds 20, which is the typical legitimate ceiling for a single shift. D2 flags role claims that are not in the allowed set, which would indicate a tampered JWT. D3 flags duplicate timestamps in the tool movement log, which would indicate replay or tampering. Alerts are displayed in an admin-only panel. A TRL-6 successor would forward these signals to Prometheus and Alertmanager.

5.8 Offline operation

A service worker caches the app shell on first visit using a cache-first strategy for static assets and a network-first strategy for API calls. If the depot Wi-Fi drops the app continues to load and the frontend falls back to its localStorage store. New records added offline are not automatically synced when the network returns. A queued sync mechanism is planned for TRL-6.

5.9 Accessibility

Two switchable themes are exposed through a small toolbar in the page header. The high-contrast theme replaces the slate colour palette with pure black backgrounds and pure white text, which meets WCAG 2.1 AA contrast 7:1. The larger-text theme raises all text by approximately 25 per cent and applies to inputs, buttons, badges and headings. Preferences persist in localStorage and apply on every page. The toolbar is itself focusable by keyboard with visible focus rings.

5.10 Print export

A print-only stylesheet hides the navigation, dashboard, and tool board when the user triggers `window.print()` from the details panel. What prints is a clean single-record job sheet suitable for clipping into a depot paper file. This was added in response to feedback from the depot manager that some inspections still need a paper record for compliance.

6. Testing

The project has 26 automated tests split across two files. The unit test file `mlService.test.js` covers riskBand thresholds, the component-index mapper, the sigmoid function, and the local inference function. The integration test file `server.test.js` boots the Express app and exercises authentication, items CRUD, JWT enforcement, and the predict endpoint. Tests run under Jest 30 and complete in under one second.

Suite	Tests	Coverage focus
mlService.test.js	15	Pure functions for inference and banding
server.test.js	13	REST API, JWT auth, bcrypt verification
Total	**28**	

Of particular value is the test that asserts a critical fault scores higher than a low fault. This catches regressions in the model coefficients or the feature ordering, both of which would otherwise be silent failures.

Manual testing was performed on Chrome 125, Edge 125, and Firefox 124 on Windows and on Chrome Mobile 125 on Android. The AR view requires camera permission and was tested under daylight, fluorescent and dim conditions. The Hiro marker was reliably detected at distances up to 60 cm with the rear camera on a Samsung Galaxy Tab A8.

7. Evaluation and TRL Assessment

7.1 Against the requirements

All four functional requirements in section 3.1 are met. The two highest-impact non-functional requirements, sub-100 ms prediction latency and WCAG 2.1 AA accessibility, are also met. The deployment requirement is met by Vercel and the test count requirement is exceeded.

7.2 Limitations

The training data is synthetic. While the feature distributions are realistic for a UK depot, the labels were generated by a hand-crafted probability function rather than observed failures. The reported model metrics therefore reflect the quality of the feature engineering on synthetic data and would need re-validation on real records before TRL-6.

The in-memory backend store resets on cold starts of the Vercel function. This is acceptable for a demonstration but would be unacceptable in production. A MongoDB Atlas migration is the obvious TRL-4 step.

The AR view uses fiducial markers rather than spatial anchors. Spatial anchors would remove the need for printed markers but require ARCore or ARKit native code, which is out of scope at TRL-3.

7.3 TRL ladder

The Technology Readiness Level scale runs from 1 (basic concept) to 9 (deployed in service). This prototype is at TRL-3 because the integrated user flows are demonstrated end-to-end in a representative environment but with simulated data and no real users. The TRL-4 step would replace synthetic data with two months of real depot records, swap in MongoDB, and add argon2id password hashing. TRL-5 would deploy to two pilot mechanics for one shift each. TRL-6 would extend to the full depot for a one-month trial.

7.4 Security analysis using STRIDE

A STRIDE pass identifies the major residual risks at TRL-3:

- **Spoofing.** Mitigated by JWT signature verification and bcrypt password hashes.
- **Tampering.** Mitigated by the hash-chain audit log for tool movements.
- **Repudiation.** Mitigated by the audit log carrying the username for each action.
- **Information disclosure.** Partly mitigated. JWT is bearer-only at TRL-3. TLS 1.3 is provided by Vercel.
- **Denial of service.** Not mitigated. Rate limiting is planned for TRL-6.
- **Elevation of privilege.** Mitigated by role-claim verification on every protected route.

7.5 Ethical considerations

The system collects mechanic names and timestamps. Under UK GDPR this is personal data and the depot would need to inform mechanics through a privacy notice at TRL-5. The synthetic training data avoids any real personal data so the prototype itself has no ethics burden.

8. Group Contribution

Member	Primary ownership
Jude	AR view, A-Frame integration, marker tuning
Abishek	Backend API, JWT, bcrypt auth, deployment
Shiar	ML training, SHAP, confusion matrix, dashboard charts
Exauce	Accessibility, service worker, print export
Mikku	Tool board, audit log, anomaly panel, tests

All members contributed to the report and presentation. Commit history is preserved on the project Git repository.

9. Conclusion and Future Work

The prototype demonstrates that an integrated AR plus ML maintenance support workflow can be assembled at TRL-3 on a small budget using free open-source libraries and a free hosting tier. The trained logistic regression achieved an F1 of 0.850 and an AUC of 0.926 on synthetic data, which is sufficient for the prototype to make plausible predictions during demonstration.

The most valuable next step is a small-scale field validation. Two months of real depot data would let the model be retrained, the synthetic-data caveat be removed, and the false-negative cost be evaluated with real stakeholder input. Beyond that, the priorities are MongoDB persistence, argon2id password hashing, and an offline write-queue for records added during a Wi-Fi outage.

10. References

1. Public Sector Bodies (Websites and Mobile Applications) (No. 2) Accessibility Regulations 2018. UK Statutory Instruments.
2. W3C. Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation, 5 June 2018.
3. Lundberg, S.M. and Lee, S.I., 2017. A unified approach to interpreting model predictions. Advances in neural information processing systems, 30.
4. Pedregosa, F. et al., 2011. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, pp.2825-2830.
5. AR.js Organization, 2024. AR.js: Augmented Reality for the Web. https://ar-js-org.github.io/AR.js-Docs/
6. Provos, N. and Mazières, D., 1999. A future-adaptable password scheme. USENIX Annual Technical Conference, FREENIX Track.
7. Jones, M., Bradley, J. and Sakimura, N., 2015. JSON Web Token (JWT). IETF RFC 7519.
8. Shostack, A., 2014. Threat modeling: Designing for security. John Wiley & Sons.
9. Karush, M. and Lewis, R., 2022. SHAP explanations for predictive maintenance in transport. Transport Research Procedia, 62, pp.483-490.
10. Vercel Inc., 2024. Vercel Functions Documentation. https://vercel.com/docs/functions